

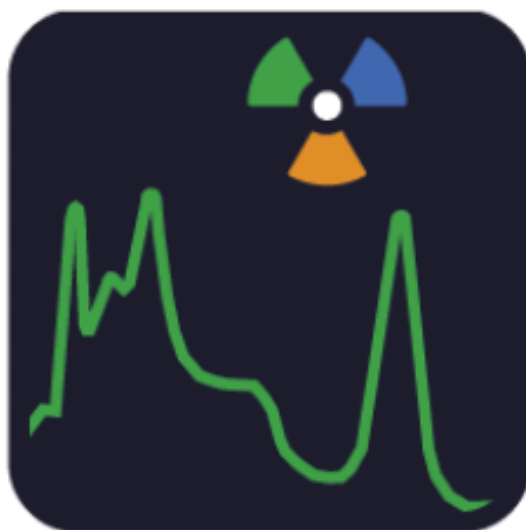
Medical Radiation Physics,
Lund University,
Sweden

October 2, 2026

LuRaCs - Reference Manual

author

Erik Ewald



LUND
UNIVERSITY



Contents

1	Getting Started	2
1.1	Introduction	2
1.1.1	LuRaCs-H3	2
1.2	Installation	2
1.2.1	With <code>pip</code>	2
1.2.2	From source	3
1.3	Connecting Devices	3
1.4	Compatible file formats	4
1.5	Package dependencies and Resources	4
2	Interface	5
2.1	Top Tabs	5
2.1.1	Spectrum	5
2.1.2	Spectrogram	8
2.1.3	Map	10
2.2	Bottom Tabs	11
2.2.1	Spectrum Info	11
2.2.2	Spectrum ROIs	12
2.2.3	Spectrogram ROIs	12
2.2.4	Real Time Values	12
2.2.5	Devices	12
2.2.6	Isotopics	12
2.2.7	Console	12
2.2.8	System Log	13
2.3	Gamma Tools	13
2.3.1	Calibration	13
2.3.2	Deconvolution	13
2.3.3	Efficiency	13
2.3.4	Resolution	13
2.4	Command Line Interface (CLI)	13
2.5	Connection Dialogs	13
2.6	Data Store	13
2.7	Settings	13
3	Examples	14
3.1	Connecting Detective X using direct ethernet	14
4	Internal Function	15
4.1	Code Structure	15
4.2	Wrapping Detectors	17
4.3	Algorithms	18
4.3.1	Peak Fitting	18
4.3.2	Deconvolution	18
4.4	File Types	18
4.4.1	Spectrum, ROI and Instrument (.xml)	18
4.4.2	Spectrogram (.db)	18
4.4.3	Detector Response Matrix (.npz)	19
4.4.4	Simple map tracks (.geojson)	20
4.5	Embedded map	21

1 Getting Started

1.1 Introduction

1.1.1 LuRaCs-H3

LuRaCs-H3 is a light weight version of the compiled program indented for system with limited storage. It excludes heavy components to reduce file size of the executable. It is approximately 250 MB smaller than the regular version. The following systems are excluded in the H3 version.

- The embedded map, position data can still be recorded.

1.2 Installation

Before installing, ensure that Python version 3.10, 3.11, 3.12 or 3.13 is installed on the system. The Python installation can be verified by opening *cmd* on windows or a Terminal on MacOS and Linux and entering `python --version` or `python3 --version`, if Python is correctly installed the version will be displayed.

1.2.1 With pip

When Python has been correctly set up, make a new folder for the install and open *cmd* or a terminal in that folder. On Windows enter the following commands in *cmd*:

```
# Make a new python environment
python -m venv .venv

# Activate the environment
.\.venv\Scripts\activate

# Install
pip install luracs

# Run
LuRaCs
```

On Linux and MacOS run the following bash commands:

```
# Make a new python environment
python3 -m venv .venv

# Activate the environment
source .venv/bin/activate

# Install
pip install luracs

# Run
LuRaCs
```

This will install all dependencies for the full version of the application and provide the start script LuRaCs.

1.2.2 From source

LuRaCs can be installed from source by cloning the GitHub repository. Installing from source requires a C/C++-compiler for compiling *Cython* extensions. When installing from source to edit the code, run `pip install -e .[dev]` to install in editable mode. This will also install development tools like `pyinstaller` and `pytest`.

```
# Clone the the GitHub repo
git clone --recursive https://github.com/EBELU/LuRaCs.git
cd LuRaCs

# Install with pip
pip install .

# Run
LuRaCs
```

1.3 Connecting Devices

Table 1.1: Currently implemented devices

Model	Manufacturer	Connections	Note
RadiaCode-1xx	Radiacode	USB, BLE	
Raysid	Raysid	BLE	
digiBase	Oretec Ametek	USB	Requires Ortec Firmware
digiDart	Oretec Ametek	USB	
DetectiveX	Oretec Ametek	NETWORK	

In order to use USB connected devices on Linux the udev-rules must be updated by adding the following permissions depending on the device.

```
# Radiacode
SUBSYSTEM=="usb", ATTR{idVendor}=="0483", ATTR{idProduct}=="f123", TAG+="uaccess",
↳  MODE="0660"

# digiBase/digiDart
SUBSYSTEM=="usb", ATTRS{idVendor}=="0a2d", GROUP="users", MODE="0666"
```

Connecting devices over the network likely requires specific to that instrument so no general guide is provided. See example 3.1 for how this can be done for the Ortec Ametek Detective X.

1.4 Compatible file formats

LuRaCs is compatible with a selection of file formats, both standard and more niche third party formats. However, LuRaCs is not primarily intended to support general spectrum analysis so if you are looking for a general analysis application consider using InterSpec¹ instead.

Table 1.2: Compatible spectrum formats.

Source	Extension	Contained Data
Radiacode	.xml	Counts, background, live time, real time, calibration
<i>Several</i>	.n42	Counts, background, live time, real time, calibration
Ortec	.spe	Counts, live time, real time, calibration
Canberra	.tka	Counts, live time, real time

Table 1.3: Compatible track formats.

Source	Extension
Radiacode	.rctrk

1.5 Package dependencies and Resources

¹*Sandialabs/InterSpec*. Sandia National Laboratories, Aug. 28, 2025. URL: <https://github.com/sandialabs/InterSpec> (visited on 08/29/2025).

2 Interface

2.1 Top Tabs

2.1.1 Spectrum

The spectrum view shows spectra as a count histograms, retrieved from a connected device or loaded from a file. It can show several spectra combined in one plot or separately. This can be changed under **Options** → **Tabbed / Combined Spectrum View**.

Background spectra can be viewed as one the three following methods *No background*, *Background overlay* and *background subtract*. *Background overlay* shows the background spectrum overlaid on top of the main spectrum. *Background subtract* does channel wise subtraction of the background from the foreground spectrum, this is a hard background subtraction and will effect ROI calculations on the spectrum. Both *Background overlay* and *background subtract* only show values of count rate.

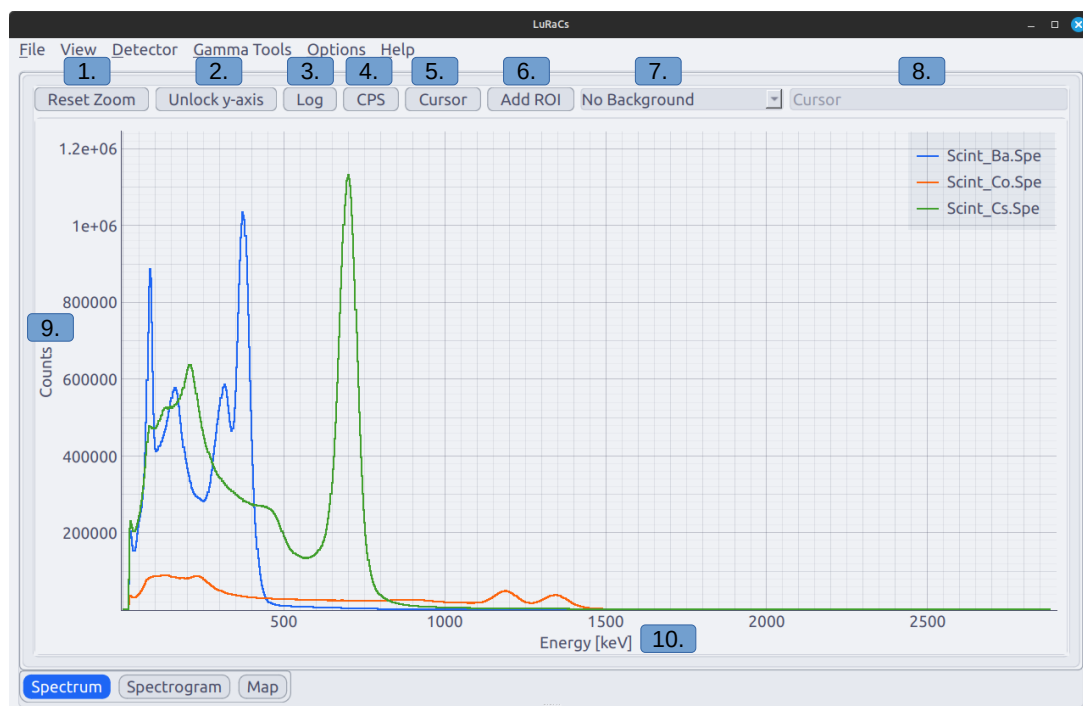


Figure 2.1: Spectra of Ba-133, C-60 and Cs-137 displayed in *Combined Spectrum View*.

1. **Rest Zoom** resets the zoom of the spectrum and locks the y-axis if unlocked.
2. **Unlock y-axis** allows the y-axis to be zoomed independently and disables autoscaling.
3. **Log** set the scale of the y-axis to logarithmic.
4. **CPS** normalises the spectrum to the live time, or real time if live time is not available but real time is.
5. **Cursor** toggles the spectrum cursor, see Figure 2.3.
6. **Add ROI** adds a customisable *region of interest* to the spectrum, see Figure 2.2.
7. Switch the use of background spectrum. Options are *No background*, *Background overlay* and *background subtract*. See the section above.
8. Display for info from the cursor, see Figure 2.3.
9. The y-axis show either total counts or time normalised counts in units of 1/s.
10. The x-axis always display energy in units of keV.

Regions of interest (ROI) are sub regions of the spectrum bounded in energy and are

useful in the analysis of spectrum peaks.

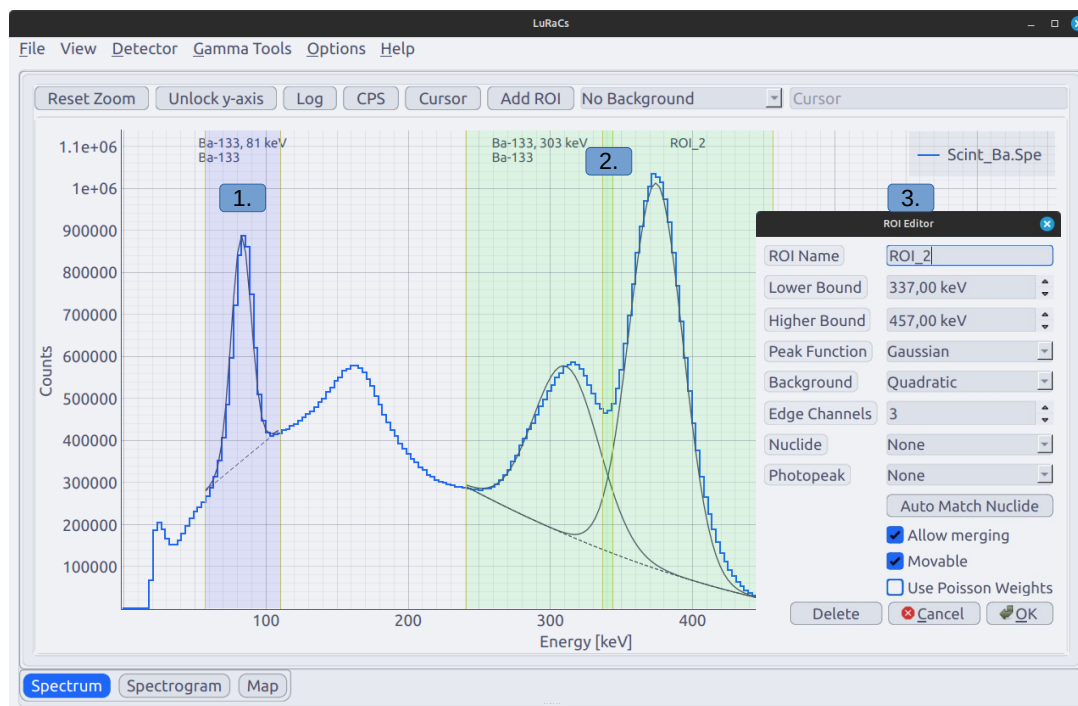


Figure 2.2: A Ba-133 spectrum with three ROIs, two are merged and fitted to the same background. The ROI editor is open for the right most ROI.

1. A ROI with one peak and a fitted Gaussian peak and linear background.
2. Two merged ROIs that fit two Gaussian peaks on a shared quadratic background.
3. The ROI editor for 'ROI_2', see Table 2.1.

Table 2.1: ROI Editor options

Field	Description
ROI Name	An alias for the ROI used for display purposes and easier identification.
Lower Bound	The lower energy bound of the ROI.
Upper Bound	The upper energy bound of the ROI.
Peak Function	The type function fitted to the peak. Options are: <i>Gaussian</i> , <i>None</i>
Background	Type of function used as peak background. Options are: <i>None</i> , <i>Linear</i> , <i>Quadratic</i>
Edge Channels	The number of channels inside of ROI from the upper and lower bounds of the ROI used to fit the background.
Nuclide	Assign a nuclide from the reference library to the ROI. Used for calculations by other tools.
Photopeak	Choose the specific photon emission associated with the peak in the ROI. Options are based on the chosen nuclide.
Auto Match Nuclide	Attempt to automatically find the Nuclide and Emission of the peak. If any nuclides are checked and shown in the spectrum the match is limited to these. Blind matches are often unreliable.
Allow Merging	Allow the ROI to merge with other ROIs with merging allowed.
Movable	If the ROI can be moved by using the mouse.
Use Poission Weights	Apply a $1/\sigma$ weight to the optimisation to force the peak to emulate a Poission distribution.

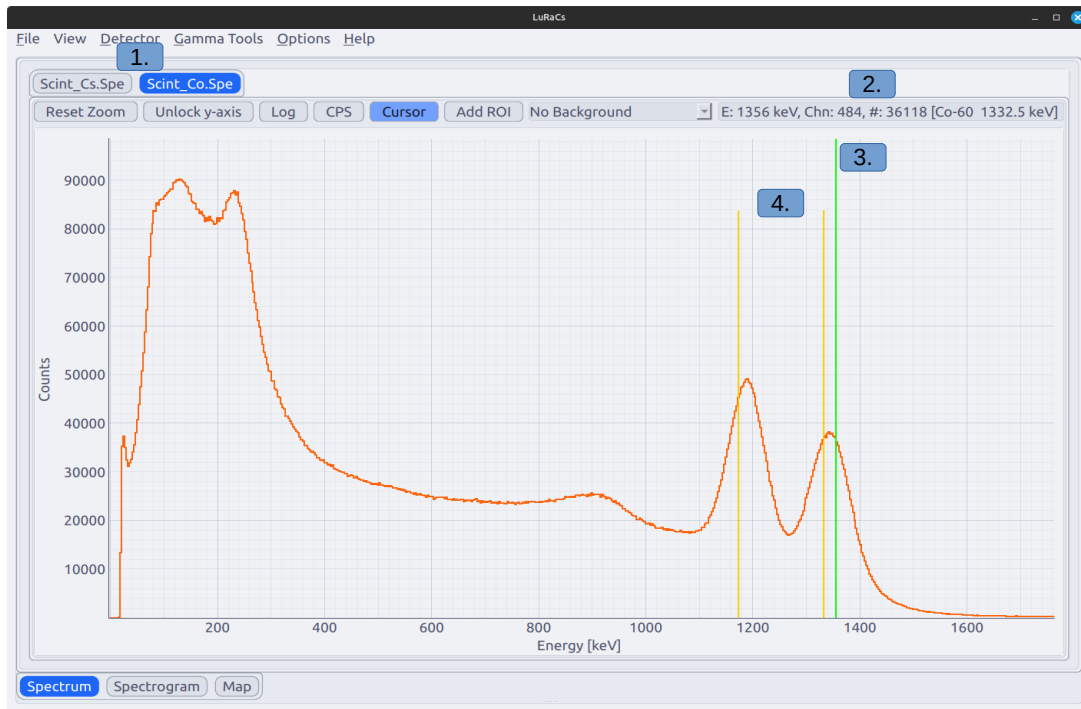


Figure 2.3: Co-60 spectrum with an active cursor.

1. Tabs for selecting spectrum in *Tabbed Spectrum View*.
2. Information about the cursor, it shows position of the cursor in units of energy, corresponding channels number, the number of counts in the channel, closest reference emission line as [*nuclide Emission Energy Decay chain if applicable*]. Channel and channel counts are only displayed in tabbed spectrum view.
3. Cursor in the spectrum.
4. Emission lines of the suggested nuclide. These are always to closest to the cursor and does not weigh intensity or anything else. Can be disabled by **View -> Spectrum -> Show Cursor Emissions**.

2.1.2 Spectrogram

A spectrogram allows time-series measurements to be conducted. A spectrogram stores data at specified time intervals and can be displayed in a waterfall plot.

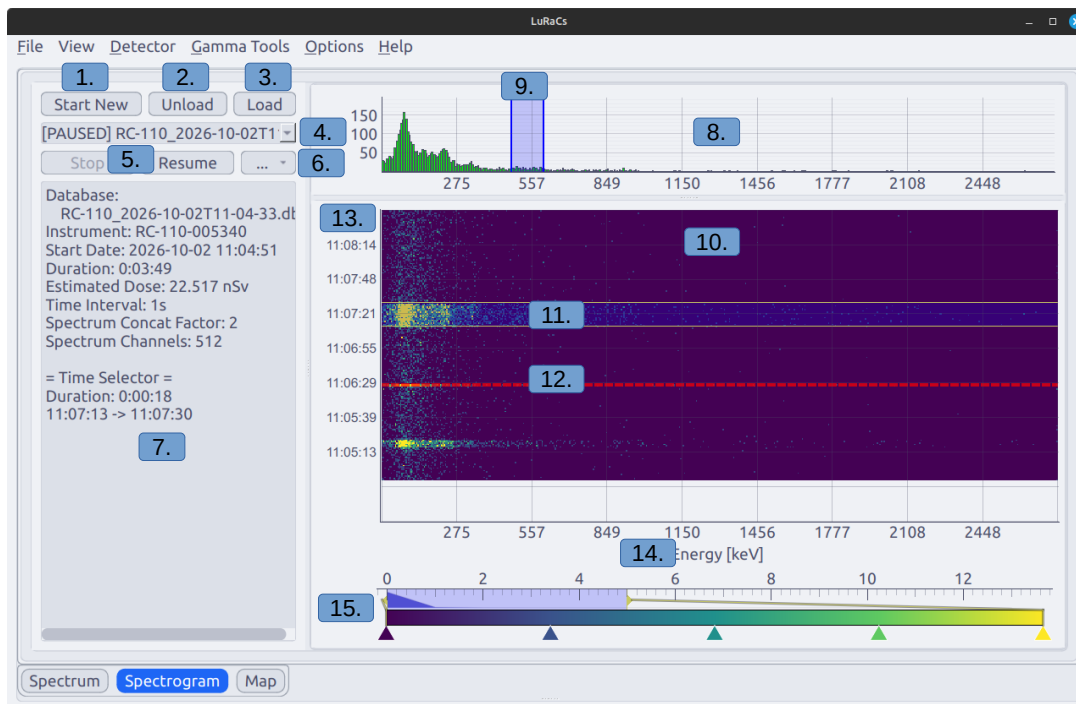


Figure 2.4: Measured spectrogram with the *Time Selector* active and one ROI.

1. Opens a dialog for starting a new spectrogram.
2. Close the viewed spectrogram.
3. Shortcut to the Data Store Spectrogram page, see section ??.
4. Selector for currently loaded spectrograms. Status is shown to the left as [ACTIVE], [LOADED] or [PAUSED].
5. Pause or Resume data collections, requires that the same instrument used collect the previous data to be connected.
6. Options menu, see ??
7. Current information stored in the spectrogram and information about the *Time Selector* if active.
8. Count histogram. Shows either total accumulated spectrum upon loading a measured spectrogram, summed channel counts inside the *Time Selector* if active or counts in the latest row.
9. ROI illustrated in the histogram, can be deleted with Mouse-2.
10. Waterfall diagram of the spectrogram.
11. *Time Selector* to study a specific time interval. Can be toggled in the options menu (6.).
12. Break line, illustrates that the time between measured points exceeds the Time Interval. Can indicate a connection and caution should be taken to verify data integrity. Can be toggled in the options menu (6.).
13. Y-axis displays the timestamps of when each row was measured.
14. X-axis display spectrum energy if calibrated.
15. Color slider for the waterfall diagram. Color map can be changed by clicking Mouse-2 on the gradient bar.

The spectrogram format is based around an SQLite database, see section 4.4.2 for further details on the format. The following data is stored for each time point in the spectrogram:

- Timestamp.
- Average count rate measured since the last time point.

- Average dose rate measured since the last time point.
- Device temperature at the time the data is written.
- Longitude at the time the data is written, if a GPS is available.
- Latitude at the time the data is written, if a GPS is available.
- Spectrum difference since the last written data point, stored as compressed binary.
- Meta data, stored as a utf-8 encoded and zlib compressed json-string. Examples of stored metadata include extended GPS data.

2.1.3 Map

Loading a map enables viewing of geographic measurements made with a spectrogram and a GPS. Maps can be loaded from online API requiring an internet connection or loaded from a *.mbiles*-file for offline use. Compatible APIs and files are offered by several vendors, the software *QGis* offers great support for exporting *.mbiles* from other formats.

Disclaimer LuRaCs does not provide or distribute map data. Users are responsible for ensuring that any map data or tile services they use with LuRaCs are obtained and used in accordance with the applicable licenses, terms, and usage restrictions.

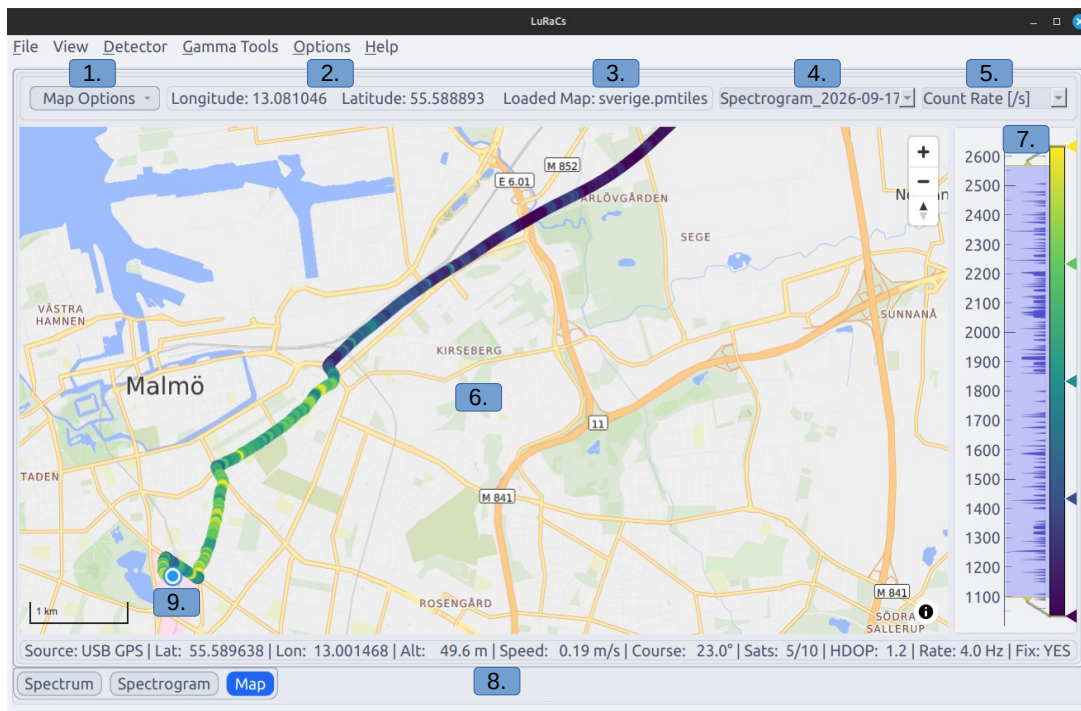


Figure 2.5: Map with a loaded track and connected GPS.

1. Options menu for loading maps and importing and exporting simple tracks.
2. Longitude and Latitude of the cursor.
3. Name of the loaded map.
4. List of loaded spectrograms, if the selected contains no geolocation data no points are displayed.
5. Selector for what data is shown. total Count Rate, Dose Rate or count rate inside a loaded ROI.
6. Main map view.
7. Color gradient selector for the track points and a histogram over measured values. Color map can be changed by clicking Mouse-2 on the gradient bar.
8. Live data from the GPS, see ??
9. Current GPS position, always displayed as a white and blue circle.

2.2 Bottom Tabs

2.2.1 Spectrum Info

Energy [keV]							
Spectrum Spectrogram Map							
Spectrum info Spectrum ROIs Spectrogram ROIs Real Time Values Detectors Isotopics Console System Log							
	Spectrum	Type	Counts	Live Time	Real Time	Calibrated	Instrument
1	RC-103-011143	Foreground	74 793	2:23:19	None	True	RC-103-011143
2	RC-103G-001618	Foreground	3 106 563	3 days, 14:29:42	None	True	None
3	RC-110-005340	Foreground	166 808	2:23:01	None	True	RC-110-005340
4	digilBase-88491	Foreground	315 789	0:12:01	0:12:02	False	None
5	DetectiveX_192.168.50.2	Foreground	178 536	0:11:53	0:11:59	True	None
6	2025-07-25-1033_Fontan.Spe	Foreground	582 238	0:10:01	0:10:03	True	None
7	HPGeEU152Corrected.TKA	Foreground	887 018	0:10:00	0:10:23	False	None

Figure 2.6

2.2.2 Spectrum ROIs

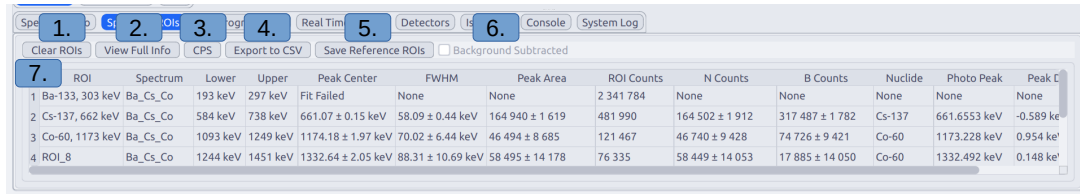


Figure 2.7

2.2.3 Spectrogram ROIs

2.2.4 Real Time Values

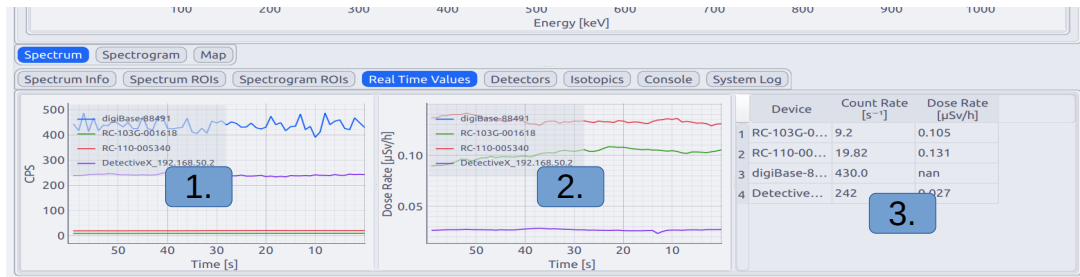


Figure 2.8

2.2.5 Devices

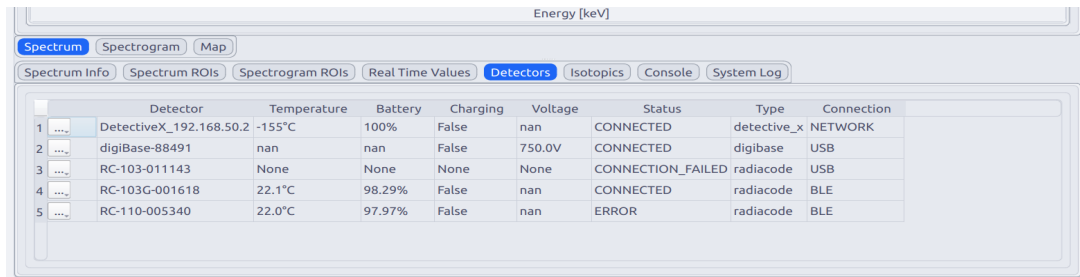


Figure 2.9

2.2.6 Isotopics

2.2.7 Console

The console allows for entering text commands to control the application and running scripts for basic automation. See subsection 2.4 for information on available commands and their usage.

2.2.8 System Log

2.3 Gamma Tools

2.3.1 Calibration

2.3.2 Deconvolution

2.3.3 Efficiency

2.3.4 Resolution

2.4 Command Line Interface (CLI)

2.5 Connection Dialogs

2.6 Data Store

2.7 Settings

The settings are divided into **Settings** which contain basic and cosmetic settings, such as what data should be imported files and what theme should the application use.

3 Examples

3.1 Connecting Detective X using direct ethernet

LuRaCs communicates with a Detective X detector using the *Mobile Field Kit* API described in the Detective X manual. Using this interface requires a network connection between the instrument and a host computer. A simple and stable way to establish such a connection is a direct ethernet cable from the instrument to the PC with no router in between. This method requires some setup before *LuRaCs* can connect to a Detective X and is described in this example.

The first step is to configure the ethernet interface on the Detective X. Go to **Settings** → **Communications** → **Ethernet**. Ensure ethernet is enabled and disable DHCP. Manually enter the IP-address and network mask. A good choice of IP is **192.168.50.2**, as long as does not interfere with another connected device, and set the network mask to **255.255.255.0**.

The second step is to configure the PC to have a static IP address on the ethernet connection. The used IP will be **192.168.50.1** and the network mask 255.255.255.0, leave gateway empty. Examples are shown for Windows 11 and Linux Mint but the principle is the same for most systems.

Once the network is configured check the connection by pinging Detective X. On most Linux systems, if the ping fails try restarting the network manager with the following command `sudo systemctl restart NetworkManager`. If that does not work, try rebooting the PC.

If the ping succeeds everything is ready. In *LuRaCs* go to **Devices** → **Connect Network** and enter the instrument IP, 192.168.50.2 in this example.

4 Internal Function

4.1 Code Structure

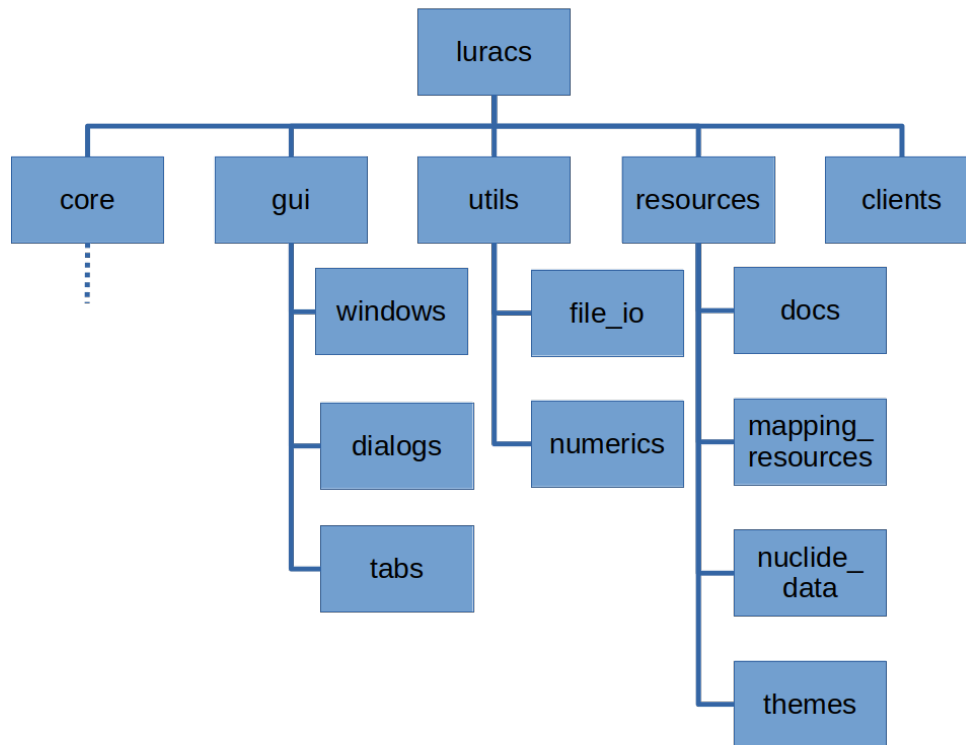


Figure 4.1: Flowchart illustrating the structure of the LuRaCs code

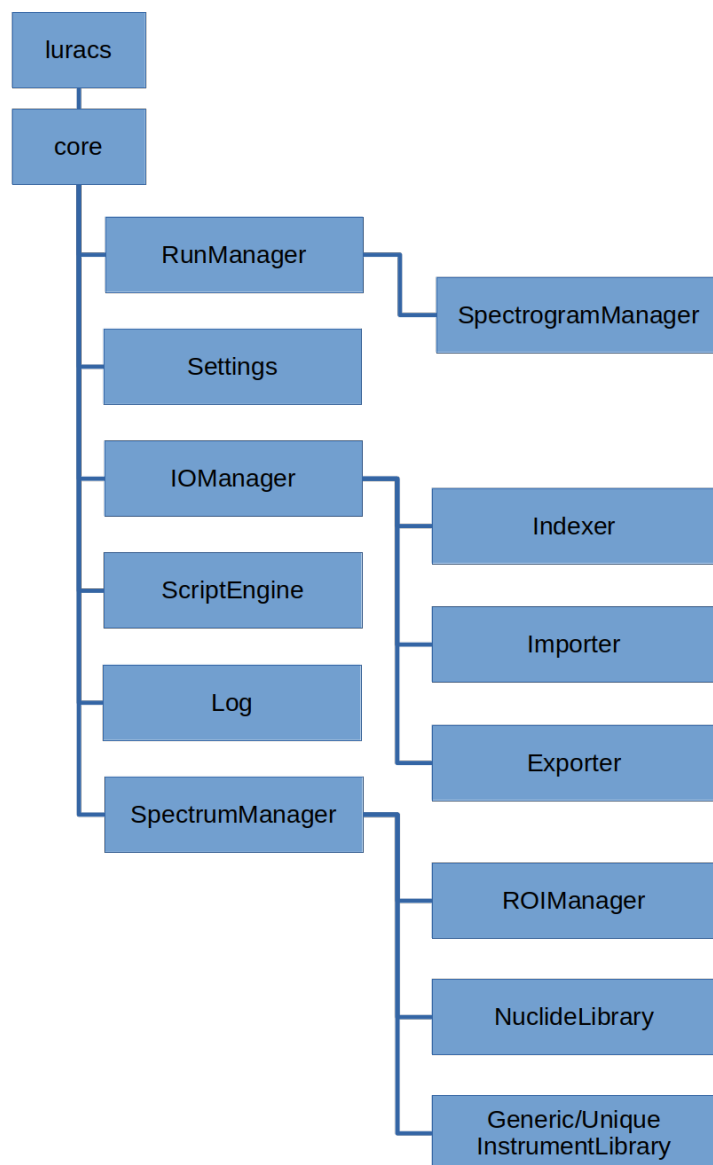
core

Figure 4.2: Flowchart of the structure of the LuRaCs core module

The core of the application is built on a number of singleton classes that manage the global state of the program. These are reachable from anywhere in the program.

Settings The settings contain modifiable parameters that affect the running of the program. It is divided in the the following parts. Settings can also be used for communication within the program as when a setting is changed using the `update_setting` function a signal is emitted from **Settings** with the group of the setting, name of the setting and the new value.

- **Appearance** contain simple settings mostly controlling appearance of the GUI and preferences for data import.
- **Paths** contain file and system paths to resources and stored data.
- **Temp** contain values that are reset upon shutdown.

- **State** saves mostly last used values for convenience and is capable of saving and loading complex data types like dequeues.
- **Advanced** contains advanced options that most users rarely need to change.

RunManager The RunManager handles connecting and sampling instruments. It owns its own thread running an async event loop, this to avoid instrument sampling being affected by events in the GUI thread and vice versa. Threadsafe communications are handled using Qt signals. The RunManager also owns the **SpectrogramManager** which tracks spectrogram and manages spectrogram ROIs.

ScriptEngine The ScriptEngine is responsible for converting text commands into actions. Like the RunManager it owns its own thread and communications are made using Qt signals. Available commands are described in the section on the CLI.

Log The Log exports a reference to the python logger "Application".

IOManager The IOManager provides functions for import and export of various files and manages indexes of internally stored file for the Data Store.

SpectrumManager The SpectrumManager is one of the most critical parts of the application. It owns all spectra, both from files and devices. Modifications of spectra are handled through it. It also owns the following components.

- **NuclideLibrary** contains loaded nuclide data and some functions for interacting with it.
- **ROIManager** handles spectrum ROIs and updates peak fits.
- **GenericInstrumentLibrary** contains generic instruments based on instrument models that can be used to store characteristic data and be used as basis for Unique-Instruments.
- **UniqueInstrumentLibrary** contains unique instruments that can be assigned to spectra. They differ from GenericInstruments in that they store information unique to each device like energy calibration.

The core also contains the helper class **Calculator** that is designed to easily assign a CPU intensive task to a separate thread and signal back the result using a Qt signals.

4.2 Wrapping Detectors

The implementation of detectors is written to be extensible and make it easy to implement new instruments. This is facilitated through inheritance from an abstract class **DeviceWrapper**. This wrapping hides the individual device implementations and LuRaCs only interact through this wrapper interface. Creating a subclass from **DeviceWrapper** automatically registers the new wrapper for use as a detector connection.

4.3 Algorithms

4.3.1 Peak Fitting

Peak fitting are based in ROIs in which a peak is assumed to be. Each ROI can contain one peak but ROIs can merge so several peaks can share the same background estimation. One a ROI is defined the fitting process is as follows.

1. Determine a number of channels starting at the ROI and counting inwards on which to base the background.
2. A polynomial is fitted using the edge channels and their corresponding counts.
3. The polynomial is evaluated for every channel in the ROI or the merged ROI region and the result subtracted from the respective channels.
4. One or the sum of several Gauassian functions are fitted to the remainder.
5. Peak characteristics are extracted from the fitted paramters.

The uncertainties of the peak fitting is calculated as follows. Firstly, the counts in channel i , N_i is assumed to follow Poisson distribution, thus $\sigma_i = \sqrt{N_i}$. The uncertainty of each channel is used as weights for the polynomial fit.

4.3.2 Deconvolution

The deconvolution method implemented is based on the *Maximum Likelihood - Expectation Maximisation* or ML-EM algorithm.

Before reconstruction the response matrices are processed and rows for the requested values are interpolated. The method for interpolation is described by Jandel et al.².

4.4 File Types

4.4.1 Spectrum, ROI and Instrument (.xml)

Spectra are stored in the XML format and is interoperable with the ANSI/IEEE N42.42 format **<empty citation>**

ROIs and Instrument data are stored as an extension to the N42 standard using the *LRC* namespace.

4.4.2 Spectrogram (.db)

Spectrograms are saved to an SQLite3 database file. One file contains one spectrogram. Saved files can be loaded and measurement resumed. A spectrogram database contains three tables described in Table 4.1, 4.2 and 4.3.

²M Jandel et al. "Decomposition of Continuum y-Ray Spectra Using Synthesized Response Matrix". In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 516.1 (Jan. 1, 2004), pp. 172–183. ISSN: 0168-9002. DOI: 10.1016/j.nima.2003.07.047.

Table 4.1: Schema of the `header` table.

Column	Type	Description
<code>id</code>	INTEGER	Identifier of the header record.
<code>created</code>	INTEGER	Creation time as a Unix timestamp.
<code>device_id</code>	TEXT	Identifier of the device producing the data.
<code>channels</code>	INTEGER	Number of spectrum channels.
<code>calibration</code>	TEXT	Calibration coefficients as a json list.
<code>concat</code>	INTEGER	Indicates whether the channels have been concatenated by a factor of 1 (not concatenated), 2, 4 or 8.
<code>save_interval</code>	REAL	Interval between data saves.
<code>meta</code>	TEXT	Additional metadata as json.

Table 4.2: Schema of the `summary` table.

Column	Type	Description
<code>id</code>	INTEGER	Identifier of the summary record.
<code>accumulated_spectrum</code>	BLOB	Accumulated spectrum data, zlib compressed array.
<code>total_duration</code>	REAL	Total duration of the recording in [s].
<code>total_dose</code>	REAL	Total accumulated dose in [μ Sv].
<code>last_update</code>	INTEGER	Timestamp of the last update as a Unix timestamp.

Table 4.3: Schema of the `spectrogram` table.

Column	Type	Description
<code>id</code>	INTEGER	Identifier of the spectrogram record.
<code>timestamp</code>	REAL	Timestamp of the measurement as a Unix timestamp.
<code>avg_cps</code>	INTEGER	Average counts per second multiplied by 1000.
<code>avg_dr</code>	INTEGER	Average dose rate as [μ Sv/s*1000].
<code>temperature</code>	INTEGER	Temperature during the measurement in [$^{\circ}$ C*1000].
<code>latitude</code>	REAL	Latitude of the measurement location.
<code>longitude</code>	REAL	Longitude of the measurement location.
<code>spectrum</code>	BLOB	Recorded spectrum during interval, zlib compressed array.
<code>meta</code>	BLOB	Additional metadata as a json string encoded with utf-8 and zlib compressed.

4.4.3 Detector Response Matrix (.npz)

Detector response matrices are stored as serialized and compressed numpy arrays (.npz) with pickle disabled. Each files contains three arrays described in Table 4.4.

Table 4.4: Contents of a response matrix .npz file.

Key	dtype	shape	Description
<code>response_matrix</code>	float32	2D (m x n)	Detector response values. First index is the simulated photon energy and second index is the normalized count bins (#bin / #fired photons)
<code>indices</code>	float32	1D (m)	Simulated photon energy of the corresponding row in <code>response_matrix</code> in keV
<code>bin_centres</code>	float32	1D (n)	Energy of the bin centres of the detector energy response in keV.

4.4.4 Simple map tracks (.geojson)

Spectrograms are the primary way to store GPS position data these are however not directly compatible with any other software without custom scripts. A spectrogram can be converted to the *GeoJSON* format, with the loss of some information, for compatibility and easier sharing. The *GeoJSON* format is generally compatible with dedicated GIS software, like *QGis*, for viewing measurements outside the embedded map.

Table 4.5: Collection-level contents of the GeoJSON mapping data file.

Key	dtype	Description
<code>type</code>	string	GeoJSON object type. Always set to "FeatureCollection".
<code>name</code>	string	Name or title of the mapping dataset.
<code>metadata</code>	object	Metadata describing the mapping session, including device information, acquisition times, and additional metadata.
<code>metadata.device_id</code>	string	Identifier of the device used to acquire the measurements.
<code>metadata.start</code>	string	ISO 8601 timestamp indicating the start of the measurement period.
<code>metadata.end</code>	string or null	ISO 8601 timestamp indicating the end of the measurement period. Set to null if unavailable.
<code>features</code>	array	Collection of georeferenced measurement points.

Table 4.6: Contents of individual measurement features in the GeoJSON mapping data. Accessed as `features.xxx`

Key	dtype	Description
<code>type</code>	string	GeoJSON feature type. Always set to "Feature".
<code>geometry</code>	object	Geographic representation of the measurement. The geometry is a GeoJSON Point.
<code>geometry.type</code>	string	Geometry type. Always set to "Point".
<code>geometry.coordinates</code>	float	Longitude and latitude of the measurement location, given in GeoJSON order as <code>[longitude, latitude]</code> .
<code>properties.timestamp</code>	string	ISO 8601 timestamp indicating when the measurement was recorded.
<code>properties.count_rate</code>	float	Measured count rate at the corresponding geographic location.
<code>properties.dose_rate</code>	float	Measured dose rate at the corresponding geographic location.
<code>properties.other_data_points</code>	on	Additional measurements points, typically produced by spectrogram ROIs, saved as a JSON-object.

4.5 Embedded map

The embedded map is rendered using *MapLibre GL JS*³ in a QWebEngine widget. The packages for MapLibre is included in the LuRaCs code and does not require any installation. MapLibre handles requesting tiles is connected to an online API it requires minimal work. Using maps offline works by using an embedded tile server run using *uvicorn* and *fastapi* to supply MapLibre with tiles. The embedded tile server runs on host 127.0.0.1 and port 8080. PMTiles are loaded using a fork of the *pmtiles* Python package, having improved cacheing and performance, incorporated in the LuRaCs code.

The code is written so that the the Python side handles most of the logic, using only simple calls to JS for creating or updating points, etc.

³*Maplibre/Maplibre-Gl-Js*. MapLibre, Sept. 21, 2026. URL: <https://github.com/maplibre/maplibre-gl-js> (visited on 09/21/2026).

References

- Jandel, M et al. “Decomposition of Continuum γ -Ray Spectra Using Synthesized Response Matrix”. In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 516.1 (Jan. 1, 2004), pp. 172–183. ISSN: 0168-9002. DOI: 10.1016/j.nima.2003.07.047.
- Maplibre/Maplibre-Gl-Js. MapLibre, Sept. 21, 2026. URL: <https://github.com/maplibre/maplibre-gl-js> (visited on 09/21/2026).
- Sandialabs/InterSpec. Sandia National Laboratories, Aug. 28, 2025. URL: <https://github.com/sandialabs/InterSpec> (visited on 08/29/2025).